DH-03: Architecture Decision Proposal and Rationale

Research Proposal and Funded Project Lifecycle Management

Institution: Horizon University Research and Innovation Office

Document purpose: Define and justify the proposed architectural decisions for the DH-03 system.

Document status: Proposed architectural baseline for team review.

Scope: Application structure, identity and access control, traceability, document storage, proposal-to-project transition, automated notifications, and business rule enforcement.

1. Purpose and Architectural Approach

1.1. Purpose of This Document

This document establishes a set of architectural decisions for the Research Proposal and Funded Project Lifecycle Management system. Its purpose is not simply to describe what the system should contain, but to explain **why the proposed architecture should work the way it does, which alternatives were considered, and how each decision follows from the project's needs**.

The architectural proposal is based on the following priorities:

- Users must access only the opportunities, proposals, documents, and projects relevant to their responsibilities and permissions.
- The system must preserve the history of information, document versions, modifications, and decisions.
- Proposals and funded projects must remain distinct while maintaining a traceable relationship.

- Documents must be stored and managed without compromising historical integrity.
- Notifications must reduce the need for users to monitor processes manually.
- The application should integrate with institutional services instead of unnecessarily replacing them.
- The solution must remain feasible to implement and demonstrate within the academic project's scope.

These priorities originate from the intended behavior of the system. The specific technologies and patterns proposed in this document are recommendations derived from those priorities, not assumptions that the institution already uses particular products or infrastructure.

1.2. How Architectural Decisions Will Be Justified

Every major decision will be evaluated through four questions:

1. **What problem must the system solve?** The decision must address a concrete functional requirement, quality attribute, or constraint.
2. **Which alternatives are available?** The proposal must consider plausible alternatives instead of presenting the first idea as the only possible solution.
3. **Why is the selected option more appropriate?** The rationale must connect the alternative's characteristics to the needs of DH-03.
4. **What are the consequences and limitations?** Every decision introduces costs, restrictions, and responsibilities that must be acknowledged.

This approach is particularly important because the project requires a defensible architecture. A technology should not be selected merely because it is popular, familiar, or associated with a large technology provider.

2. Architectural Decision 1: Application Structure

AD-01. Adopt a modular monolithic architecture as the initial deployment model.

Decision status: Proposed.

2.1. Problem and Motivation

The system manages several related areas: research opportunities, proposals, reviews, institutional approvals, sponsor decisions, funded projects, documents, notifications, and closure activities.

These areas have different responsibilities, but they are closely connected through the research lifecycle. A funding decision may lead to project activation, a document approval may allow a proposal to advance, and a project change may generate audit records and notifications.

The architecture must separate these responsibilities without introducing unnecessary operational complexity.

The project also has an academic constraint: the team must deliver a coherent solution within a semester and demonstrate an end-to-end workflow. Therefore, the architecture should make responsibilities explicit while keeping deployment, integration, testing, and maintenance manageable.

2.2. Alternatives Considered

Alternative A: A conventional monolithic application

A conventional monolith packages the system as one application. It can be straightforward to deploy and operate, but without deliberate modular boundaries, different business responsibilities may become tightly coupled.

For DH-03, this could lead to document management, access control, notifications, and lifecycle rules becoming mixed together. Changes in one area could unintentionally affect another, and different developers could implement inconsistent versions of the same business rule.

A conventional monolith is feasible, but an unstructured implementation does not adequately address the need for clear responsibilities and traceability.

Alternative B: A modular monolithic application

A modular monolith runs as one deployable application but divides its functionality into modules with explicit responsibilities and interfaces.

This allows the proposal lifecycle, project lifecycle, document management, audit, identity, and notifications to remain logically separate while avoiding the need to deploy and operate each module independently.

This alternative supports the project's need for organization without introducing distributed communication as an additional problem.

Alternative C: A microservices architecture

Microservices would allow individual services to be deployed and scaled independently. They could be useful if the system had independently scaling workloads, separate operational teams, strong isolation requirements between services, or an established distributed platform.

However, the current project information does not establish that these conditions exist. A microservices design would introduce additional concerns such as service-to-service communication, distributed failures, network security, deployment coordination, observability, and consistency across service boundaries.

For example, activating a funded project could involve proposal records, document references, audit records, and notification events. Splitting these operations across services would require additional mechanisms to manage partial failures and consistency.

These challenges are solvable, but their cost is not justified merely because the system has several functional areas.

2.3. Rationale for the Decision

The modular monolith is the preferred initial option because it balances two requirements expressed by the project:

- The architecture must separate responsibilities clearly.
- The implementation must remain feasible within the academic scope.

It provides logical separation without requiring each responsibility to become an independently deployed service.

It also supports the user's intention to centralize lifecycle management. The system can coordinate proposal approvals, document versioning, project activation, and audit records through well-defined application operations.

The recommendation does not mean that the system will never require distributed services. It means that such complexity should only be introduced when concrete requirements justify it.

2.4. Proposed Module Boundaries

The application will contain the following logical modules:

- **Identity and authorization:** user identity references, permissions, and resource access rules.
- **Opportunities and proposals:** opportunity registration, proposal preparation, review, approval, submission records, and sponsor decisions.
- **Funded projects:** activation, milestones, deliverables, changes, and closure.
- **Document management:** document metadata, versions, storage references, and integrity information.
- **Audit:** historical records of significant operations and decisions.
- **Notifications:** event processing, delivery status, and scheduled reminders.

These modules will interact through defined application interfaces rather than directly modifying each other's internal data structures.

2.5. Consequences

Benefits

- Responsibilities remain clear and independently understandable.
- The team can divide implementation work by module.
- Business rules can be centralized and tested.
- Deployment and demonstration remain relatively simple.
- Future separation into services remains possible if justified.

Costs and limitations

- Modules share a deployment and may share a database.
- A failure in the application can affect multiple modules.
- Independent scaling of individual modules is limited.
- The team must actively enforce module boundaries to prevent the monolith from becoming tightly coupled.

2.6. Conditions That Could Change This Decision

The team should reconsider the deployment model if new evidence establishes independent scaling requirements, strict service isolation, separate deployment schedules, or an institutional platform that already requires distributed services.

Without such evidence, a modular monolith is the more proportionate choice for the current scope.

3. Architectural Decision 2: Identity and Authorization

AD-02. Separate user authentication from application authorization and enforce access through both roles and resource relationships.

Decision status: Proposed.

3.1. Problem and Motivation

The system will contain information with different access requirements. Some opportunities may be discoverable by multiple researchers, while proposal documents, reviews, and project records may be restricted to specific participants.

The user has explicitly identified the need to ensure that each person can access only the opportunities and proposals relevant to them.

This requirement cannot be satisfied by assigning a general role alone. Two users may both be principal investigators while having permission to access different proposals.

The architecture must therefore distinguish between establishing a user's identity and determining what that user may do.

3.2. Alternatives Considered

Alternative A: Email-based identification and authorization

Email is convenient for communication and can be used to identify a contact. However, an email address may change and does not, by itself, establish that a person is authorized to access a particular proposal.

Using email as the sole basis for authorization would also confuse two separate concerns: identity and communication.

Therefore, email should be treated as contact information, not as the definitive authorization mechanism.

Alternative B: Role-based access control only

Role-based access control (RBAC) assigns permissions according to roles such as PI, reviewer, coordinator, or administrator.

It provides a useful foundation, but it cannot express every resource-specific relationship on its own.

For example, a reviewer may be permitted to evaluate proposals generally, but should only access proposals assigned to them. Similarly, a team member should not automatically see every project simply because they are a member of a research team.

RBAC alone is therefore insufficient.

Alternative C: Resource relationships only

A relationship-based model determines access according to a user's relationship with a resource, such as being a proposal owner or project member.

This directly addresses the user's requirement, but a purely relationship-based approach may become unnecessarily complicated when defining broad operational permissions.

For example, the system must distinguish between who may approve a proposal and which particular proposal that person is allowed to approve.

Alternative D: Combined role and relationship-based authorization

This model combines general permissions with resource-specific relationships.

A role determines whether a user can perform a type of operation. A relationship determines whether the user can perform that operation on the particular resource.

This is the recommended alternative because it addresses both dimensions of the access problem.

3.3. Rationale for the Decision

The combined model follows directly from the user's intention that access depend on both the person's responsibilities and the opportunities or proposals with which they are associated.

For example:

- A PI may be allowed to create proposals but may only edit proposals they are authorized to manage.
- A reviewer may have permission to submit evaluations but only for assigned proposals.
- A coordinator may manage assigned opportunities without automatically gaining unrestricted access to every confidential document.
- A project member may access their project without being granted access to unrelated proposals.

This model provides more precise access control than either role-based permissions or resource relationships alone.

3.4. Authentication Strategy

If Horizon University has an available institutional identity provider, the application should delegate authentication to it.

OpenID Connect is a candidate integration standard because it allows an application to rely on an external identity provider rather than maintaining a separate institutional password system.

The application would maintain its own user record linked to a stable identity-provider identifier.

This recommendation follows from the user's intention to integrate with existing institutional systems instead of replacing them.

However, the availability and compatibility of the university's identity provider must be confirmed. The architecture does not assume that a particular identity platform is already installed.

If institutional integration is unavailable in the academic environment, a limited demonstration authentication mechanism may be used. It must preserve the same authorization model and should not be represented as a production institutional integration.

3.5. Proposed Access Rules

The following rules establish the initial authorization model:

1. Opportunity visibility will be explicitly defined as public within an authorized community or restricted to designated users.
2. Access to an opportunity will not automatically grant access to every proposal associated with it.
3. Proposal access will depend on the user's role and relationship with that proposal.
4. Reviewer access will be limited to assigned reviews.
5. Project access will be evaluated independently from proposal access.
6. Administrative permissions will be separated from ordinary research access.
7. Authorization will be enforced by the server on every protected operation.

The final role definitions and permissions must be checked against the functional requirements. The proposed rules establish a direction without inventing institutional policies that have not been specified.

3.6. Consequences

Benefits

- Supports restricted proposal information.
- Prevents broad roles from granting excessive access.
- Separates institutional identity from application permissions.
- Makes access rules easier to test.
- Supports future identity-provider integration.

Costs and limitations

- Requires explicit modeling of user-resource relationships.
- Requires careful handling of permission changes.
- Adds authorization checks to protected operations.
- Depends on confirmation of institutional identity integration.

3.7. Validation Scenario

Given two PIs with different restricted proposals, when one PI attempts to retrieve the other PI's proposal through a direct application request, the server must deny access even if the user knows the proposal identifier or URL.

This scenario makes the security decision testable rather than merely describing the system as secure.

4. Architectural Decision 3: Historical Traceability

AD-03. Preserve document versions and maintain an append-only application audit history for significant business operations.

Decision status: Proposed.

4.1. Problem and Motivation

The user explicitly wants to preserve the history of information, document versions, modifications, and decisions.

This requirement is especially important because the system manages proposals that undergo revisions and approvals before being submitted to a sponsor. Later, an accepted proposal may become the basis of a funded project.

If the system simply overwrites the current document or updates a decision field, it may become impossible to determine what was originally submitted, what changed, and which version was approved.

The architecture must therefore preserve historical evidence separately from current operational state.

4.2. Alternatives Considered

Alternative A: Store only the latest version

This approach minimizes storage and implementation complexity. However, it loses the ability to reconstruct previous versions and is inconsistent with the user's explicit traceability requirement.

It is rejected.

Alternative B: Store document versions but no audit history

Versioning preserves the contents of files, but it does not necessarily explain who approved a version, when it was submitted, or why a project changed.

It addresses only part of the problem.

Alternative C: Store document versions and relevant audit events

This approach preserves both the historical content and the events that explain how the system reached its current state.

It is the recommended option because it supports the user's intention to maintain a complete and understandable history.

4.3. Rationale for the Decision

Document versioning and auditing answer different questions:

- Versioning answers: *What did the document contain at a particular point in time?*
- Auditing answers: *Who performed an operation, when did it occur, and what was its result?*

Neither is a substitute for the other.

For example, retaining three versions of a proposal does not establish which one was submitted to the sponsor. An audit record identifying the submission and referencing the exact version provides that missing connection.

The system will therefore preserve both the historical artifacts and the events that relate them to business decisions.

4.4. Audit Record Design

Significant events will record:

- Event identifier.
- User or system actor.
- Timestamp.
- Operation type.
- Affected entity and identifier.
- Outcome.
- Relevant version or object reference.
- Reason or comment when required.

Events will cover proposal submissions, approvals, rejections, document changes, access changes, project activation, milestone updates, change requests, and closure.

The audit log will be append-only through normal application functionality. Ordinary users and business operations will not be able to edit or delete previous audit events.

This is an application-level protection, not a claim that database administrators or infrastructure operators could never alter the underlying records. Stronger tamper-evidence requirements would need additional controls and explicit justification.

4.5. Document Versioning Design

Each logical document will have one or more version records.

A version record will include:

- Document identifier.
- Version identifier.
- Associated proposal or project.
- File storage reference.
- Uploader and timestamp.
- Version status.
- File format and size.
- Content hash.
- Reference to the previous version, when applicable.

Uploading a replacement will create a new version. It will not silently change the content of an existing version.

4.6. Why Not Store a Complete Snapshot of the Entire System After Every Change?

Full system snapshots could preserve historical states, but they would duplicate large amounts of data and make document storage particularly inefficient.

The system does not need to duplicate every unrelated entity whenever a proposal changes.

Instead, it will preserve versioned documents and audit records for significant operations. Where an exact historical business state is necessary, the design can introduce explicit snapshots of the relevant data.

This is a more targeted approach to the user's goal of traceability without unnecessarily multiplying the storage requirements.

4.7. Consequences

Benefits

- Preserves historical evidence.
- Supports reconstruction of decisions.
- Prevents submitted documents from being silently replaced.
- Improves accountability.
- Supports the transition from proposal to project.

Costs and limitations

- Requires more storage.
- Requires retention and access policies.
- Increases the complexity of document lifecycle management.
- Requires clear definitions of which operations must be audited.

The additional storage and implementation effort are justified because historical preservation is a core requirement, not an optional enhancement.

5. Architectural Decision 4: Relational Database and Object Storage

AD-04. Separate structured business data from document contents using a relational database and object storage.

Decision status: Proposed.

5.1. Problem and Motivation

The system manages two fundamentally different types of information.

The first consists of structured records, such as opportunities, users, proposals, project milestones, approvals, and document metadata.

The second consists of document contents, such as proposal files, supporting attachments, and project documents.

The user has already identified the need for a relational database for structured information and separate storage for files.

The architectural question is how to implement that separation while preserving consistency and traceability.

5.2. Alternatives Considered

Alternative A: Store files directly in the relational database

A relational database can store binary file contents. This simplifies some transactional scenarios because the metadata and file can be managed together.

However, large document contents can increase database storage requirements, backup sizes, and operational overhead. File retrieval and retention also become more closely tied to database management.

This option may be appropriate for small systems or specific requirements, but it is not the preferred initial design given the user's intention to separate structured data from documents.

Alternative B: Store everything in object storage

Object storage is appropriate for files, but it is not a suitable replacement for the relational management of complex business relationships and lifecycle transitions.

The system needs structured relationships between users, proposals, approvals, projects, and document versions. A relational database better supports those requirements.

Alternative C: Use a relational database for metadata and object storage for files

The relational database manages structured records and relationships, while object storage holds document contents.

This is the recommended option because it matches the different responsibilities of the two data types.

5.3. Rationale for the Decision

The system needs to answer structured questions such as:

- Which proposals belong to an opportunity?
- Who is authorized to review a proposal?
- Which project originated from a particular proposal?
- Which document version was approved?
- Which milestones remain incomplete?

These relationships benefit from relational constraints and structured queries.

Document contents have different characteristics. They can be large, need independent retention policies, and may be accessed less frequently as they become historical.

Separating them permits the system to manage structured data and files according to their respective needs.

PostgreSQL is proposed as the initial relational database because it supports relational constraints and transactions. It is a recommendation, not a requirement derived directly from the scenario.

An object storage service should be selected only after its versioning, access controls, cost, retention, and retrieval characteristics have been evaluated.

5.4. Consistency Between the Database and Object Storage

The database and object storage will not share a single ordinary transaction. The system must therefore handle failures between the two operations.

For example, a file upload may succeed while the database operation fails, or the database may record an upload attempt before the object becomes available.

The document upload workflow will therefore:

1. Create a controlled upload record.
2. Store the file in object storage.
3. Verify the resulting object and its integrity.
4. Finalize the version record in the database.
5. Expose the document version as available only after verification.

Failed or abandoned uploads will be identified for retry or cleanup according to a defined recovery policy.

The exact workflow may vary with the selected storage service, but the architectural requirement remains the same: **a document must not be presented as a valid historical version until its contents have been stored and verified.**

5.5. Storage Tiers and Archival

The user proposed evaluating storage tiers according to access frequency, including options similar to those offered by AWS.

The architectural decision is to support the possibility of archival, but not to select a particular tier or automatically move files into one before the requirements are known.

The choice depends on:

- Expected access frequency.
- Required retrieval time.
- Storage and retrieval costs.
- Retention requirements.
- Availability requirements.
- Whether historical documents must be immediately accessible during reviews or audits.

Using a lower-cost archival tier may reduce storage expenses but can introduce retrieval delays or additional charges. Therefore, the architecture must not assume that the cheapest storage class is automatically the best option.

The storage policy will be selected after the team establishes the relevant usage patterns and constraints.

5.6. Consequences

Benefits

- Clear separation between structured records and file contents.
- More suitable storage management for each data type.
- Independent document retention and archival policies.
- Easier management of document metadata and version references.

Costs and limitations

- Requires coordination between two storage systems.
- Introduces partial-failure scenarios.
- Requires additional upload verification and recovery logic.
- May require extra retrieval time or costs for archived documents.

These costs follow directly from the decision to preserve documents independently while maintaining reliable references to them.

6. Architectural Decision 5: Proposal and Funded Project Separation

AD-05. Represent proposals and funded projects as distinct business entities connected through an explicit activation process.

Decision status: Proposed.

6.1. Problem and Motivation

The user explicitly identified that a proposal must not be confused with a funded project.

A proposal represents a request for funding. A funded project represents the execution of an initiative after a favorable funding decision and the completion of the required activation conditions.

Their information overlaps, but their responsibilities differ.

The proposal contains preparation history, reviews, approvals, submitted versions, and sponsor decisions. The project contains execution information, milestones, deliverables, changes, and closure records.

The architecture must preserve this distinction while ensuring that the relationship between the two remains traceable.

6.2. Alternatives Considered

Alternative A: Use one entity with a changing status

Under this design, a proposal would become a project simply by changing its status.

This is superficially simple, but it mixes two different lifecycle models. Fields required during proposal preparation may have different meanings from fields required during execution.

It could also make historical interpretation difficult if project updates overwrite information associated with the approved proposal.

This alternative is not recommended.

Alternative B: Create a separate project without a permanent relationship to the proposal

This separates the operational records, but loses the explicit connection between the funded project and the proposal that justified it.

The system would have difficulty proving which proposal and document version formed the basis of the project.

This alternative does not meet the traceability requirement.

Alternative C: Create a separate project linked to the originating proposal and accepted version

This design separates operational information while retaining the historical relationship.

It is the recommended option.

6.3. Rationale for the Decision

The user's objective is not to discard the proposal when funding is awarded. The objective is to continue managing the funded work while preserving the history that explains how it originated.

A separate project entity allows operational information to evolve without rewriting the original proposal record.

For example, project dates may change, a deliverable may be revised, or a new project member may be assigned. Those operational changes should not alter the historical record of what was originally approved.

At the same time, the project must retain a reference to the accepted proposal version so that authorized users can retrieve the relevant evidence.

6.4. Activation Process

Project activation will be an explicit business operation that:

1. Confirms that a favorable funding decision has been recorded.

2. Identifies the exact proposal version used as the project's basis.
3. Records the required funding and activation information.
4. Creates the funded project entity.
5. Links the project to its originating proposal and opportunity.
6. Records the activation actor and timestamp.
7. Generates the corresponding audit and notification events.

If required information or approvals are missing, the project must remain in a preparation state rather than being treated as fully activated.

6.5. What Should Be Copied and What Should Be Referenced?

The architectural rule is:

Preserve the proposal history in the proposal domain, reference the accepted proposal version from the project, and copy only the information needed for independent project operations.

This rule follows from the user's intention to maintain historical information without unnecessarily duplicating the entire proposal lifecycle.

The project should not contain a second independent copy of every proposal version, review, and decision.

Instead:

- The proposal retains its document history, reviews, approvals, and sponsor decision.
- The project retains a permanent reference to the originating proposal and accepted version.
- Operational information that must evolve independently is maintained as project-specific data.
- Where the original approved value must remain distinguishable from a later operational value, both are preserved explicitly.

This design avoids indiscriminate duplication while keeping the relationship between the original approval and subsequent execution understandable.

6.6. Handling Later Revisions

A document correction after submission will create a new version. It will not overwrite the version previously submitted.

If the sponsor requests revisions, the system will record the request and the resulting version.

If funding is awarded, the version used to establish the project must be explicitly identified. The system must not assume that the most recently uploaded version is the accepted one.

This is necessary because document recency and sponsor acceptance are different facts.

6.7. Consequences

Benefits

- Preserves the distinction between preparation and execution.
- Keeps the original proposal history intact.
- Supports project-specific operational changes.
- Maintains a traceable relationship between funding approval and execution.
- Prevents the wrong document version from becoming the project's historical basis.

Costs and limitations

- Requires a controlled activation workflow.
- Introduces relationships between multiple entities.
- Requires rules for copied information and historical values.
- Requires explicit handling of sponsor decisions and revisions.

These costs are justified by the need to preserve the proposal's history while allowing the funded project to evolve independently.

7. Architectural Decision 6: Event-Driven Notifications

AD-06. Generate notifications from business events and process delivery asynchronously, with a separate mechanism for scheduled reminders.

Decision status: Proposed.

7.1. Problem and Motivation

The user wants to reduce the need for continuous manual monitoring.

The system should inform responsible users when an action requires their attention, such as a review assignment, approval, correction request, funding decision, or approaching deadline.

Not all notifications arise in the same way. Some are triggered by an operation, while others depend on the passage of time.

The architecture must distinguish between these cases.

7.2. Alternatives Considered

Alternative A: Manual notifications

Users would be responsible for checking the system and informing others when action is required.

This does not address the user's intention to reduce manual monitoring and makes timely communication dependent on individual behavior.

It is rejected as the primary approach.

Alternative B: Send emails directly inside each business operation

For example, the proposal approval function could save the approval and immediately send an email.

This approach is easy to understand initially, but it couples business operations to email delivery. A temporary email service failure could interfere with the operation, or the operation could succeed while the notification is lost.

It also encourages different modules to implement inconsistent notification logic.

Alternative C: Record events and process notifications asynchronously

Business operations record the event, while a background worker handles delivery.

This separates business processing from notification delivery and allows failures to be retried.

It is the recommended option.

7.3. Rationale for the Decision

The user's requirement is to make the system proactive rather than dependent on manual monitoring.

Event-driven notifications allow the system to react to meaningful changes. Asynchronous delivery prevents external email availability from becoming a necessary condition for every business operation.

For example, an approval should be recorded successfully even if the email service is temporarily unavailable. The notification can remain pending until the service becomes available.

7.4. Transactional Outbox

The application will use the Transactional Outbox pattern for relevant events.

When a business operation is committed, its pending notification event will be written to the same relational database transaction.

A background worker will then process pending events and attempt delivery through the available email service.

This pattern is recommended because it reduces the risk of losing a notification between the successful database operation and the external email request.

It does not guarantee exactly-once delivery. The worker must handle retries, duplicate-prevention mechanisms, and ambiguous delivery outcomes.

7.5. Scheduled Reminders

Event-driven notifications do not replace reminders for future deadlines.

A separate scheduled process will identify approaching or overdue dates and generate reminder events.

For example, a proposal may remain unchanged while its submission deadline approaches. No new business operation is required to trigger the reminder, so the system must evaluate dates independently.

The reminder process will use the same notification pipeline, avoiding a separate email implementation for each reminder type.

7.6. Recipients and Confidentiality

Recipients will be determined from the affected entity's relationships and the applicable notification rules.

Notifications will not grant access to documents or proposals. Recipients must still authenticate and pass the authorization checks before accessing the referenced content.

Email content will be limited to what is necessary to communicate the event. Confidential documents will not be attached by default.

This follows from the user's original emphasis on access restrictions and the fact that the system handles research information.

7.7. Consequences

Benefits

- Reduces manual monitoring.
- Separates business operations from email delivery.
- Supports retries after temporary failures.
- Provides a common mechanism for events and reminders.
- Makes notification history easier to inspect.

Costs and limitations

- Requires background processing.
- Requires tracking of delivery attempts and failures.

- Requires duplicate handling.
- Requires explicit rules for recipients and reminder timing.

The pattern is justified by the need for reliable notifications, not by a general preference for asynchronous systems.

8. Architectural Decision 7: Controlled Business State Transitions

AD-07. Enforce lifecycle transitions through explicit business operations rather than unrestricted status updates.

Decision status: Proposed.

8.1. Problem and Motivation

The system manages a lifecycle in which the completion of one stage determines whether another stage can begin.

A proposal must undergo the required review and approval process before submission. A funded project must not be activated without the appropriate funding decision and required information.

If users or modules can freely modify status fields, the system may enter inconsistent states. For example, a proposal could be marked as submitted without identifying the submitted document version.

The architecture must therefore make lifecycle transitions explicit and enforce their conditions.

8.2. Alternatives Considered

Alternative A: Allow direct status changes

This approach is simple to implement but does not guarantee that required conditions have been satisfied.

It is unsuitable for a system where approvals, submissions, and funding decisions have operational consequences.

Alternative B: Centralize transition rules in application operations

Each meaningful transition is implemented as a business operation that checks permissions, validates conditions, changes the state, and records the result.

This is the recommended option because it makes lifecycle behavior explicit and testable.

Alternative C: Introduce a separate workflow engine

A workflow engine may be valuable if the institution requires configurable workflows, complex branching, extensive timers, or frequent process changes by non-developers.

However, the available requirements do not yet establish that level of workflow complexity. Introducing a dedicated engine would add another component and operational dependency.

The initial architecture will therefore implement lifecycle rules in the application layer. A workflow engine can be reconsidered if concrete requirements justify it.

8.3. Rationale for the Decision

Controlled transitions support the user's goals of preserving history, preventing unauthorized operations, and maintaining the distinction between proposals and funded projects.

For each transition, the system will:

1. Authenticate and authorize the actor.
2. Verify the current state.
3. Validate the transition conditions.
4. Perform the business operation.
5. Record the result in the audit history.
6. Generate relevant notification events.

For example, activating a project will require a favorable funding decision and an explicitly identified proposal version. These conditions cannot be reliably enforced if the transition is implemented as a simple status update.

8.4. Consequences

Benefits

- Prevents invalid lifecycle transitions.
- Makes business rules testable.
- Supports consistent auditing and notifications.
- Reduces contradictions between modules.

Costs and limitations

- Requires a defined state model.
- Requires explicit transition operations.
- Requires the team to maintain and validate business rules as requirements evolve.

These costs are necessary because the system's value depends on preserving the integrity of its lifecycle.

9. How the Decisions Work Together

The previous decisions are not independent technology choices. They form a connected architecture.

Consider the following scenario:

A PI prepares a proposal for a research opportunity. The system verifies the PI's permissions and stores the proposal metadata in the relational database. The associated document is stored in object storage and registered as a version.

A reviewer is assigned to the proposal. The assignment is recorded as a business operation and an audit event. A notification event is stored for asynchronous processing.

The reviewer requests corrections. A new document version is uploaded, preserving the earlier version. The proposal progresses only after the required review and approval conditions have been satisfied.

When the proposal is submitted to the sponsor, the system records the exact submitted version. A later upload cannot silently change the historical record of that submission.

If the sponsor awards funding, the system records the decision and initiates the controlled project activation process. A new funded project is created and linked to the originating proposal and accepted version.

The project then manages its own milestones, deliverables, changes, and closure activities. These operations generate audit events and notifications without overwriting the original proposal history.

This scenario demonstrates why the decisions reinforce one another:

- Authorization protects the information.
- Versioning preserves the documents.
- Audit records preserve the sequence of operations.
- Controlled transitions protect the lifecycle.
- Proposal-project separation preserves the distinction between approval and execution.
- Event-driven notifications reduce manual monitoring.
- The modular monolith coordinates these responsibilities without requiring distributed services.

10. Quality Attributes and Their Architectural Justification

The architectural decisions must be evaluated against the qualities that matter to the system.

Quality attribute	Need in DH-03	Architectural response	Trade-off
Security	Restrict access to relevant opportunities, proposals, documents, and projects.	Authentication separated from authorization; role and resource relationship checks.	More explicit permission modeling and testing.

Traceability	Reconstruct changes, submissions, approvals, and funding decisions.	Audit records, document versioning, and explicit entity relationships.	Additional storage and implementation effort.
Data integrity	Prevent inconsistent relationships and invalid lifecycle transitions.	Relational constraints and controlled business operations.	More validation logic.
Maintainability	Keep different responsibilities understandable and manageable.	Modular monolith with defined module boundaries.	Module boundaries must be enforced consistently.
Operational resilience	Prevent temporary email failures from losing notifications.	Transactional outbox and asynchronous retries.	Background processing and delivery-state management.
Performance	Manage structured information and potentially large files efficiently.	Relational database for structured data and object storage for files.	Coordination between separate storage systems.
Evolvability	Integrate with existing institutional services and accommodate future needs.	Defined integration boundaries and modular responsibilities.	External integrations may require adapters and configuration.

The table describes the expected relationship between the architecture and the desired qualities. It does not claim that these qualities have already been measured or proven.

The team should define measurable quality scenarios for the final Evaluation 2 package. For example, the security scenario can specify that a PI's unauthorized request must be denied, while the traceability scenario can require retrieval of the exact submitted version after subsequent revisions.

11. Assumptions, Constraints, and Decisions Still Requiring Evidence

A defensible architecture must distinguish between decisions supported by the current problem and details that remain unknown.

The following items require confirmation:

11.1. Institutional Identity

The proposal recommends delegating authentication to an existing identity provider. The university's actual provider, integration availability, and supported protocol have not been established.

Therefore, OIDC is a proposed integration standard, not a confirmed institutional requirement.

11.2. Final Authorization Rules

The combined role and resource relationship model follows from the need to restrict access. However, the exact permissions for each institutional role must be confirmed against the functional requirements.

11.3. Retention and Archival Policies

The need to preserve document history is established by the user's objectives. The required retention periods, permitted deletions, archival tiers, and maximum retrieval times remain undefined.

No specific AWS storage class should be selected until these requirements are understood.

11.4. Sponsor Decisions and Activation Conditions

The system must record sponsor decisions and distinguish proposals from funded projects. The exact evidence required for a favorable decision and the complete set of activation conditions must be validated against the lifecycle requirements.

11.5. Notification Rules

Event-driven notifications and scheduled reminders address the stated need to reduce manual monitoring. The precise recipients, deadlines, reminder intervals, and escalation rules must be established from the requirements.

11.6. Scale and Availability

The current information does not establish a required number of users, transaction volume, availability target, or disaster recovery objective.

Consequently, the architecture does not justify microservices, multi-region deployment, or a particular cloud provider on the basis of scale or availability claims that have not been provided.

These are not reasons to avoid architectural decisions. They are boundaries on what can currently be justified.

12. Priority Proofs of Concept

The following proofs of concept should validate the architectural mechanisms with the greatest effect on the system's correctness.

PoC 1: Resource-Based Access Control

Objective: Demonstrate that the authorization model prevents access to unrelated restricted proposals.

Test: Authenticate as one PI and attempt to retrieve another PI's restricted proposal through a direct application request.

Expected result: The server denies access regardless of whether the requester knows the resource identifier or URL.

Architectural decision validated: AD-02.

PoC 2: Document Versioning and Historical Reconstruction

Objective: Demonstrate that document revisions do not destroy historical evidence.

Test: Upload a proposal document, create a new version, and retrieve the version associated with an earlier submission.

Expected result: Both versions remain distinguishable, and the earlier submitted version can still be retrieved by an authorized user.

Architectural decisions validated: AD-03 and AD-04.

PoC 3: Proposal-to-Project Activation

Objective: Demonstrate that a funded project is created independently while preserving its historical origin.

Test: Record a favorable funding decision and activate a project using a specified accepted proposal version.

Expected result: The project is created as a distinct entity, retains its relationship to the proposal, and identifies the exact version used as its basis.

Architectural decisions validated: AD-05 and AD-07.

PoC 4: Notification Reliability

Objective: Demonstrate that temporary email service failures do not require the original business operation to be repeated.

Test: Generate a notification event, simulate an email service failure, and retry processing.

Expected result: The event remains available for processing, delivery attempts are recorded, and the system applies duplicate-prevention measures where possible.

Architectural decisions validated: AD-06.

These proofs of concept are deliberately narrow. They validate the architectural assumptions most likely to affect correctness before the team commits to a larger implementation.

13. Implementation Priorities

The implementation should follow the dependencies between the architectural decisions.

1. Define the domain entities, their relationships, and the proposal and project lifecycle states.
2. Establish the identity integration approach and initial authorization policies.
3. Implement structured data persistence and document metadata management.
4. Implement object storage integration and document versioning.

5. Implement audit records and controlled lifecycle transitions.
6. Implement the proposal-to-project activation operation.
7. Implement event persistence and asynchronous notification processing.
8. Implement scheduled reminders.
9. Execute the priority proofs of concept.
10. Document the resulting evidence, unresolved risks, and required architectural adjustments.

This order ensures that access control, data integrity, and historical preservation are addressed before the system adds more extensive automation.

14. Final Architectural Position

The proposed architecture is based on a central principle: **the system must preserve the integrity and history of the research lifecycle while making the process more controlled, transparent, and less dependent on manual monitoring.**

The proposed decisions are:

- **AD-01:** Use a modular monolith because the system requires separated responsibilities but does not yet demonstrate a need for independently deployed services.
- **AD-02:** Combine role-based permissions with resource-specific relationships because users with the same institutional role may have access to different opportunities, proposals, and projects.
- **AD-03:** Preserve document versions and audit significant operations because current-state data alone cannot reconstruct historical submissions and decisions.
- **AD-04:** Separate relational business data from document contents because the two data types have different storage, querying, integrity, and retention needs.
- **AD-05:** Model proposals and funded projects as distinct entities because approval history must remain intact while project execution evolves independently.
- **AD-06:** Use event-driven notifications, asynchronous processing, and scheduled reminders because manual monitoring is insufficient and email failures must not undermine business operations.
- **AD-07:** Enforce lifecycle transitions through explicit business operations because the integrity of the process depends on validating conditions before changing states.

Each decision has a specific rationale, acknowledged trade-offs, and a proposed validation method. The choices are not presented as universal best practices; they are recommendations derived from the needs and constraints currently established for DH-03.

The remaining Evaluation 2 work should connect these decisions to the official requirements, quality scenarios, architecture views, domain model, security design, and validation evidence. If new requirements contradict one of the proposed decisions, the team should revisit the decision explicitly rather than silently changing the architecture.

The goal is not to make the architecture look sophisticated. The goal is to make every major design choice explainable, traceable to a system need, and defensible in front of the professor.